

Individual Contribution Report

Mert Ozustun

2022400192

Group 35

1.Task & Contributions

My primary role was integrating the Recovery Manager into the existing four-layer DBMS engine and making the whole system work end to end. Selcen implemented the Recovery Manager module itself, and I connected it to the rest of the project and got crash recovery working in practice. Specifically:

- **Integration/wiring:** Wired the Recovery Manager into archive.py so the layers are constructed bottom-up and the three-phase recovery (Analysis, Redo, Undo) runs automatically on startup before any input is processed. Connected it to the Disk Space Manager (dedicated wal.log with explicit fsync), the Buffer Manager (write-ahead ordering before page writeback), and the Query Processor (transaction begin/commit driving the log).
- **Checkpointing & log buffering:** Hooked the periodic fuzzy checkpoint trigger (checkpoint_interval) and the log-buffer flush (log_buffer_size) into the runtime flow, and ensured the master record was read/written correctly.
- **Debugging:** Traced and fixed cross-layer issues so committed transactions survive a crash and uncommitted ones roll back, including recovery across a fuzzy checkpoint. I also refined parts of the Recovery Manager while integrating it.
- **Configuration & tooling:** Handled the configuration file (data_dir, checkpoint_interval, log_buffer_size) and set up the test workflow (run.txt, test_cases/).

2.Collaboration & Challenges

Selcen and I split the work between the core recovery algorithm and its integration, and coordinated through Git (including resolving a merge of our branches). The main challenge was getting the write-ahead ordering correct across the boundary between the Buffer Manager and the

Recovery Manager (recovery bypasses the buffer pool and talks to disk directly), so I had to make sure log records reach stable storage before the corresponding pages and that file paths stayed consistent after moving storage files into `data_dir`. Reproducing crashes and reasoning about the on-disk state at recovery time was the hardest part to debug.

3.Self-Assessment

Through this project I learned how ARIES recovery actually works in practice — the WAL discipline, fuzzy checkpoints, and the Analysis/Redo/Undo phases — and how to debug state that spans multiple layers. I contributed strongly on the integration and debugging side. An area to improve is (e.g. writing automated tests earlier / understanding the indexing layer more deeply), which would have let me catch some integration bugs sooner.

4.Use of AI

I used Claude (via Claude Code) as an assistant while integrating and debugging the project. I used it in three main ways:

- Configuration: I asked it to help wire the `data_dir` setting through the Disk Space Manager and to check `config.json` against the project specification, so the configuration parameters were complete and used consistently.
- Debugging: When recovery did not behave as expected, I described the symptoms and asked it to help me reason through the write-ahead logging order and the Analysis/Redo/Undo flow, which helped me locate the cross-layer bugs faster.
- Understanding: I asked conceptual questions about ARIES recovery and fuzzy checkpoints to make sure my integration matched the intended behavior.

For every suggestion, I reviewed the change, adapted it to our own code, and verified it myself by running the test cases. I did not accept any code without understanding it.